

Trabajo práctico 2

Introducción

Inciso 1

¿Cuál es la función de la capa de aplicación?

Respuesta:

Proporcionar la interfaz y los protocolos necesarios para que los procesos de las aplicaciones de usuario (que se ejecutan en sistemas finales distintos) puedan comunicarse e intercambiar mensajes a través de la red.



Tip

La capa de aplicación solo se ejecuta en los sistemas terminales.

Inciso 2

Si dos procesos deben comunicarse:

1. ¿Cómo podrían hacerlo si están en diferentes máquinas?
2. Y si están en la misma máquina, ¿qué alternativas existen?

Inciso 2.1

Los procesos de dos sistemas terminales diferentes se comunican entre sí intercambiando mensajes a través de la red de computadoras. Un proceso emisor crea y envía mensajes a la red; un proceso receptor recibe estos mensajes y posiblemente responde devolviendo otros mensajes.

¿Qué necesitan saber para encontrarse?

- La **dirección IP** del host
- El **número de puerto** de destino

Inciso 2.2

Cuando los procesos se ejecutan en el mismo sistema terminal, pueden comunicarse entre sí mediante sistemas de comunicación inter-procesos,

aplicando reglas gobernadas por el sistema operativo del sistema terminal.

Alternativas:

- **Memoria compartida:** El S.O. permite que ambos procesos lean y escriban directamente en una misma zona reservada de la memoria RAM.
- **Tuberías:** Canales de comunicación donde un proceso escribe datos por un extremo y el otro proceso los lee por el otro extremo de forma secuencial.
- **Sockets de dominio local:** Sockets especiales que funcionan de manera interna en el software para comunicarse de forma sumamente rápida, sin que los datos tengan que salir a la tarjeta de red física.

Inciso 3

Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la “vida cotidiana” y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

Respuesta

En el **modelo Cliente/Servidor**, existe un host siempre activo denominado **servidor**, que tiene una dirección IP fija y conocida, y se encarga de recibir y dar servicio a las solicitudes de muchos otros hosts llamados **clientes**, los cuales inician la comunicación

Ejemplo en la vida cotidiana: El servicio en un **restaurante**. El mesero (servidor) está siempre en un local fijo esperando a que entren comensales. El cliente (comensal) es quien entra e inicia la interacción haciendo un pedido (solicitud); el mesero procesa el pedido y le trae la comida (respuesta).

Ejemplo de sistema informático: Un ejemplo clásico es la Web, en la que un servidor web siempre activo sirve las solicitudes de los navegadores que se ejecutan en los hosts clientes.

Otro modelo que conozco, podría ser el **P2P**. En esta, la aplicación explota la comunicación directa entre parejas de hosts conectados de forma intermitente, conocidos como pares (peers). Los pares no son propiedad del proveedor del servicio, sino que son computadoras de escritorio y portátiles

controladas por los usuarios, encontrándose la mayoría de los pares en domicilios, universidades y oficinas.

Inciso 4

Describe la funcionalidad de la entidad genérica “Agente de usuario” o “User agent”.

Respuesta:

La línea de cabecera User-agent: especifica el agente de usuario, es decir, el tipo de navegador que está haciendo la solicitud al servidor. Esta línea de cabecera resulta útil porque el servidor puede enviar versiones diferentes del mismo objeto a los distintos tipos de agentes de usuario

Inciso 5

¿Qué son y en qué se diferencian HTML y HTTP?

Respuesta

HTTP: Hypertext Transfer Protocol (HTTP) (o Protocolo de Transferencia de Hipertexto en español) es un protocolo de la [capa de aplicación](#) para la transmisión de documentos hipermedia, como HTML.

HTML: Lenguaje de Marcado de Hipertexto, del inglés HyperText Markup Languagees. Es el componente más básico de la Web. Es un estándar para el **formato de documentos**. Es el lenguaje de marcado que se utiliza para definir el contenido, el significado y la estructura de una página web

Uno es un protocolo por el cual se transmiten los datos, y el otro es la representación en sí de los datos dentro de la web.

Inciso 6

HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado “Formato de mensaje HTTP”, Kurose).

1. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información? ¿Para qué sirven las cabeceras?

2. ¿Cuál es su formato? (Ayuda:

<https://developer.mozilla.org/es/docs/Web/HTTP/Headers>)

3. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso /index.html. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.

Inciso 6.1

- **En un mensaje de requerimiento (solicitud):** La primera línea se denomina **línea de solicitud** (*request line*). Está compuesta por tres campos separados por espacios:
 1. **Método:** La acción que se desea realizar (ej. GET , POST , HEAD , PUT , DELETE).
 2. **URL:** La ruta del objeto que se está solicitando (ej. /unadireccion/pagina.html).
 3. **Versión de HTTP:** La versión que utiliza el cliente (ej. HTTP/1.1).
- **En un mensaje de respuesta:** La primera línea se denomina **línea de estado** (*status line*). Está compuesta por tres campos separados por espacios:
 1. **Versión del protocolo:** (ej. HTTP/1.1).
 2. **Código de estado:** Un número de tres dígitos que indica el resultado de la solicitud (ej. 200 , 301 , 404).
 3. **Frase de estado:** Una descripción textual corta y explicativa del código (ej. OK , Moved Permanently , Not Found).

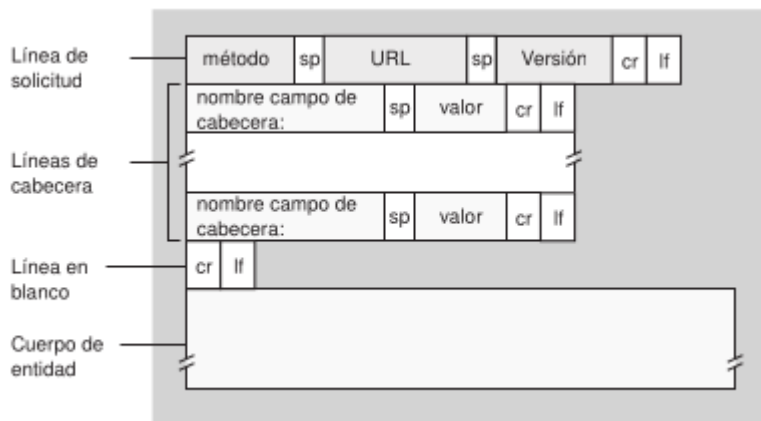
¿Para qué sirven las cabeceras? Las líneas de cabecera sirven para enviar **metadatos e información de control adicional** entre el cliente y el servidor sin alterar el contenido principal (cuerpo de entidad).

- **En la solicitud:** Permiten al cliente identificarse (con User-agent), especificar el servidor de destino (Host), indicar preferencias de idioma (Accept-language) o gestionar la persistencia de la conexión (Connection).
- **En la respuesta:** Permiten al servidor indicar el tipo y tamaño de archivo enviado (Content-Type , Content-Length), la fecha de generación

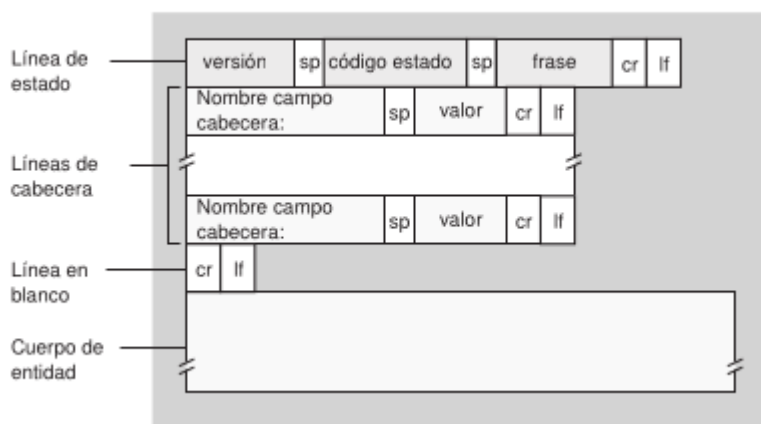
(Date), cuándo se modificó el recurso por última vez (Last-Modified) o configurar parámetros de sesión como las cookies (Set-cookie).

Inciso 6.2

Formato Genérico de Solicitud:



Formato genérico de la respuesta:



Inciso 6.3

Para solicitar el recurso `/index.html` al servidor `www.misitio.com` utilizando la versión **HTTP 1.1** desde el cliente **curl/7.74.0**, la información obligatoria y recomendada que debe enviarse mediante encabezados es:

- Host (Obligatorio)
- User-agent (recomendado)
- Accept(opcional/recomendado): Indica al servidor qué formatos de datos es capaz de procesar el cliente

Mensaje de requerimiento:

```
1 GET /index.html HTTP/1.1
2 Host:
3 User-Agent:
4 Accept:
```

Inciso 7

Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).

Nota: [Documentación de curl](#)

Respuesta

curl es una herramienta de línea de comandos diseñada para transferir datos hacia o desde un servidor a través de la red. Su principal característica es que opera sin requerir interacción del usuario, lo que la hace ideal para entornos de terminal y automatización.

Características principales:

- **Soporte multiprotocolo:** Aunque su uso más común es con HTTP y HTTPS para interactuar con la web, soporta una lista masiva de protocolos incluyendo FTP, SCP, SMTP, LDAP y MQTT, apoyándose en su motor subyacente, `libcurl`.
- **Prueba y depuración de APIs:** Permite estructurar peticiones HTTP complejas directamente desde la consola, definiendo el método (GET, POST, PUT, DELETE), inyectando cabeceras (*headers*) y enviando cuerpos de datos (*payloads*).

Parámetro	Que hace
-X	Se utiliza para especificar un método de solicitud personalizado al comunicarse con un servidor. Sirve para indicar métodos específicos como <code>PUT</code> , <code>DELETE</code> o comandos de tecnologías relacionadas como WebDAV (<code>PROPFIND</code> , <code>COPY</code> , <code>MOVE</code>), reemplazando el método por defecto que es <code>GET</code> . Esta opción <i>únicamente</i> cambia la palabra (el verbo HTTP) en la petición, pero no altera la forma en que <code>curl</code> se comporta

Parámetro	Que hace
-H	Es para para agregar, modificar o eliminar múltiples encabezados. Si se especifica un encabezado con el mismo nombre que uno que <code>curl</code> usaría internamente, tu encabezado lo sobrescribirá. Para eliminar un encabezado interno, proporcionarlo sin contenido a la derecha de los dos puntos (por ejemplo, <code>-H "Host:"</code>). Si se necesita enviar un encabezado personalizado sin valor, la cadena debe terminar con un punto y coma (por ejemplo, <code>-H "X-Custom-Header;"</code>).
-I	envía una petición HTTP HEAD . El servidor responde únicamente con las cabeceras (el código de estado 200 OK, el servidor Apache, las fechas, etc.) y omite por completo descargar el cuerpo del documento (el HTML).
-s	Cuando hacés una petición, <code>curl</code> te muestra una tabla con la barra de progreso, la velocidad, el peso y los tiempos de descarga. Al agregarle <code>-s</code> , oculta toda esa información de estado y solo te escape el dato puro que fuiste a buscar (por ejemplo, el JSON de una API o el texto de una página).

Inciso 8

Ejecute el comando `curl` sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

1. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (`>`) del comando `curl` a un archivo con extensión `html` y abrirlo con un navegador.
2. ¿Cómo funcionan los atributos `href` de los tags `link` e `img` en `html`?
3. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?
4. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando `curl` ejecutado previamente.

Inciso 8.1

Realizó un solo requerimiento, y recibió el contenido de un archivo .html, en caso de usar > se guardaría en un archivo. El mismo archivo cuenta con este contenido:



Inciso 8.2

El href como tal es:

- Link sí funciona con **href**, y sirve para especificar la URL del recurso externo (como un CSS) que el navegador debe descargar para dar formato a la página.
- En el caso de imagen, no funciona con `href`, sino con **src** (*source*), que le indica al navegador la ruta de la imagen que debe renderizar en pantalla.

Inciso 8.3

No, un solo requerimiento no alcanza.

¿Por qué? Porque HTTP solo transfiere **un objeto por cada mensaje de respuesta**. El archivo base HTML que descargamos al principio solo contiene texto y las referencias (direcciones) a los otros objetos, pero no las imágenes en sí.

Inciso 8.4

Para obtener la página completa se necesitan un total de 8 requerimientos HTTP.

Esto se debe a que la página está constituida por **8 objetos individuales** (1 archivo HTML base, 2 CSS, 2 Javascript y 3 imágenes), y HTTP transfiere **un único objeto por cada mensaje de solicitud/respuesta**.

- **Navegador Web (realiza 8 requerimientos):** Primero hace una petición para descargar el HTML base. Tras recibirlo, el navegador procesa el archivo, encuentra los 7 objetos adicionales que están referenciados mediante sus URLs (los CSS, Javascript e imágenes) y **realiza automáticamente los 7 requerimientos restantes** para completar y mostrar de forma correcta la página.
- **Comando curl (realiza 1 requerimiento):** Solo ejecuta el primer requerimiento de la lista para obtener el recurso exacto de la URL que le ingresaste (el archivo HTML base). Al ser una herramienta de consola básica y no un navegador, **no interpreta el código HTML** ni descarga de manera automática los archivos adicionales referenciados en el texto.

Inciso 9

Ejecute a continuación los siguientes comandos:

```
curl -v -s www.redes.unlp.edu.ar > /dev/null
```

```
curl -I -v -s www.redes.unlp.edu.ar
```

1. ¿Qué diferencias nota entre cada uno?
2. ¿Qué ocurre si en el primer comando se quita la redirección a /dev/null?
¿Por qué no es necesaria en el segundo comando?
3. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?

Ejecución de ambos comandos:

```

redes@debian:~$ curl -v -s www.redes.unlp.edu.ar > /dev/null
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sun, 30 Aug 2026 17:49:07 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
{ [4898 bytes data]
* Connection #0 to host www.redes.unlp.edu.ar left intact
redes@debian:~$ curl -I -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> HEAD / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Date: Sun, 30 Aug 2026 17:49:28 GMT
Date: Sun, 30 Aug 2026 17:49:28 GMT
< Server: Apache/2.4.56 (Unix)
Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
Accept-Ranges: bytes
< Content-Length: 4898
Content-Length: 4898
< Content-Type: text/html
Content-Type: text/html
<
* Connection #0 to host www.redes.unlp.edu.ar left intact

```

Inciso 9.1

El primer comando realiza una petición **GET** (el comportamiento por defecto de curl), la cual le pide al servidor tanto las cabeceras como el cuerpo del mensaje (el código HTML de la página). El segundo comando, al usar el parámetro `-I`, realiza una petición **HEAD**. Esta petición le indica al servidor que solo devuelva las cabeceras, omitiendo por completo el cuerpo del mensaje.

Inciso 9.2

Si sacamos `> /dev/null` en el primer comando, todo el código HTML de la página (los 4898 bytes que menciona la línea) se imprimirá de golpe en la terminal, ensuciando la vista. La redirección a `/dev/null` sirve justamente

para "tirar a la basura" ese cuerpo y que solo se vean las cabeceras gracias al modo verbose (-v).

Inciso 9.3

- **En el requerimiento (las líneas con >):** Viajaron **3** cabeceras (Host , User-Agent y Accept).
- **En la respuesta (las líneas con <):** Viajaron **7** cabeceras (Date , Server , Last-Modified , ETag , Accept-Ranges , Content-Length y Content-Type).
- Esto es idéntico para ambos comandos

Tip

la línea `< HTTP/1.1 200 OK` es la "Línea de estado" (Status-Line), no se cuenta como cabecera.

Inciso 10

¿Qué indica la cabecera Date?

Respuesta:

La línea de cabecera **Date:** indica el día, la fecha y la hora exacta en la que se creó la respuesta HTTP y fue enviada por el servidor

- No especifica cuándo se creó o modificó el objeto (archivo) que estás solicitando.
- Representa únicamente el momento en el que el servidor recupera el objeto de su sistema de archivos, lo inserta en el mensaje de respuesta y lo envía a la red.

Inciso 11

En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

Respuesta:

En HTTP 1.0 el cliente lo sabe porque se cierra activamente la conexión TCP inmediatamente después de enviar el archivo. Al detectar el cierre físico del

canal de comunicación, el cliente tiene la certeza de que el objeto ha sido transferido en su totalidad.

En HTTP 1.1 dado que la conexión TCP se mantiene abierta para enviar más solicitudes, el cliente no puede depender del cierre de la conexión. En su lugar, el servidor debe incluir obligatoriamente la cabecera **Content-Length** en la respuesta, especificando el tamaño exacto del objeto en bytes (así como se ve en la foto del inciso 9). El cliente va contando los bytes recibidos y, al alcanzar el número indicado, sabe que el objeto está completo

Inciso 12

Investigue los distintos tipos de códigos de retorno de un servidor web y su significado.

Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

Respuesta

Las respuestas se sacaron de [MDN DOCS](#)

Las respuestas se agrupan en cinco clases:

1. Respuestas informativas (100 – 199),
2. Respuestas satisfactorias (200 – 299),
3. Redirecciones (300 – 399),
4. Errores de los clientes (400 – 499),
5. y errores de los servidores (500 – 599).

Para desarrollar como respuesta, haremos desde la 200 en adelante

Respuestas 2XX

Los más comunes son:

[200 OK](#)

La solicitud ha tenido éxito. El significado de un éxito varía dependiendo del método HTTP.

[201 Created](#)

La solicitud ha tenido éxito y se ha creado un nuevo recurso como resultado de ello. Ésta es típicamente la respuesta enviada después de una petición PUT.

202 Accepted

La solicitud se ha recibido, pero aún no se ha actuado. Es una petición "sin compromiso", lo que significa que no hay manera en HTTP que permite enviar una respuesta asíncrona que indique el resultado del procesamiento de la solicitud. Está pensado para los casos en que otro proceso o servidor maneja la solicitud, o para el procesamiento por lotes.

203 Non-Authoritative Information

La petición se ha completado con éxito, pero su contenido no se ha obtenido de la fuente originalmente solicitada, sino que se recoge de una copia local o de un tercero. Excepto esta condición, se debe preferir una respuesta de 200 OK en lugar de esta respuesta.

204 No Content

La petición se ha completado con éxito pero su respuesta no tiene ningún contenido, aunque los encabezados pueden ser útiles. El agente de usuario puede actualizar sus encabezados en caché para este recurso con los nuevos valores.

205 Reset Content

La petición se ha completado con éxito, pero su respuesta no tiene contenidos y además, el agente de usuario tiene que inicializar la página desde la que se realizó la petición, este código es útil por ejemplo para páginas con formularios cuyo contenido debe borrarse después de que el usuario lo envíe.

Respuestas 3XX

Algunas de ellas son:

300 Multiple Choice

Esta solicitud tiene más de una posible respuesta. User-Agent o el usuario debe escoger uno de ellos. No hay forma estandarizada de seleccionar una de las respuestas.

301 Moved Permanently

Este código de respuesta significa que la URI del recurso solicitado ha sido cambiado. Probablemente una nueva URI sea devuelta en la respuesta.

302 Found

Este código de respuesta significa que el recurso de la URI solicitada ha sido cambiado temporalmente. Nuevos cambios en la URI serán agregados en el futuro. Por lo tanto, la misma URI debe ser usada por el cliente en futuras solicitudes.

303 See Other

El servidor envía esta respuesta para dirigir al cliente a un nuevo recurso solicitado a otra dirección usando una petición GET.

304 Not Modified

Esta es usada para propósitos de "caché". Le indica al cliente que la respuesta no ha sido modificada. Entonces, el cliente puede continuar usando la misma versión almacenada en su caché.

Respuestas 4XX

Los más comunes son:

400 Bad Request

Esta respuesta significa que el servidor no pudo interpretar la solicitud dada una sintaxis inválida.

401 Unauthorized

Es necesario autenticar para obtener la respuesta solicitada. Esta es similar a 403, pero en este caso, la autenticación es posible.

403 Forbidden

El cliente no posee los permisos necesarios para cierto contenido, por lo que el servidor está rechazando otorgar una respuesta apropiada.

404 Not Found

El servidor no pudo encontrar el contenido solicitado. Este código de respuesta es uno de los más famosos dada su alta ocurrencia en la web.

405 Method Not Allowed

El método solicitado es conocido por el servidor pero ha sido deshabilitado y no puede ser utilizado. Los dos métodos obligatorios, GET y HEAD , nunca deben ser deshabilitados y no deberían retornar este código de error.

409 Conflict

Esta respuesta puede ser enviada cuando una petición tiene conflicto con el estado actual del servidor.

429 Too Many Requests

El usuario ha enviado demasiadas solicitudes en un periodo de tiempo dado.

Errores 5XX

500 Internal Server Error

El servidor ha encontrado una situación que no sabe cómo manejarla.

501 Not Implemented

El método solicitado no está soportado por el servidor y no puede ser manejado. Los únicos métodos que los servidores requieren soporte (y por lo tanto no deben retornar este código) son GET y HEAD .

502 Bad Gateway

Esta respuesta de error significa que el servidor, mientras trabaja como una puerta de enlace para obtener una respuesta necesaria para manejar la petición, obtuvo una respuesta inválida.

503 Service Unavailable

El servidor no está listo para manejar la petición. Causas comunes puede ser que el servidor está caído por mantenimiento o está sobrecargado. Hay que tomar en cuenta que junto con esta respuesta, una página usuario-amigable explicando el problema debe ser enviada.

504 Gateway Timeout

Esta respuesta de error es dada cuando el servidor está actuando como una puerta de enlace y no puede obtener una respuesta a tiempo.

505 HTTP Version Not Supported

La versión de HTTP usada en la petición no está soportada por el servidor.

Inciso 13

Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

1. ¿Qué información brinda la primera línea de la respuesta?
2. ¿Cuántos encabezados muestra la respuesta?
3. ¿Qué servidor web está sirviendo la página?
4. ¿El acceso a la página solicitada fue exitoso o no?
5. ¿Cuándo fue la última vez que se modificó la página?
6. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?

Comando ejecutado

```
redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 19:02:03 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 4898
Content-Type: text/html
```

Inciso 13.1

La primer línea indica:

- Versión HTTP
- Código de la respuesta
- Mensaje si fue Ok o no

Inciso 13.2

Muestra 7 encabezados:

- Date, como la fecha y hora en la que el servidor devuelve la respuesta
- Server, como el servidor que maneja la petición
- Ultima modificación, como ultima vez que se modificó el recurso en el servidor
- ETag, Es un identificador o "huella digital" única para la versión específica de ese recurso.
- Accept-ranges, indica si el servidor soporta solicitudes parciales de contenido (descargas por rangos de bytes), lo que permite pausar y reanudar descargas.
- Content-length, como el peso exacto en bytes del contenido solicitado.
- Content-type, Define el formato o tipo de medio del recurso que se solicitó

Inciso 13.3

Como bien indica la foto

```
redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 19:02:03 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
```

Es un servidor Apache versión 2.4.56

Inciso 13.4

Sí, el código de respuesta y su mensaje de descripción indican que si

```
redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
```

Inciso 13.5

```
redes@debian:~$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 19:02:03 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-517457bd04100"
```

El Domingo 19 de marzo de 2023 a las 19:04:46

Inciso 13.6

```
redes@debian:~$ curl -v -z "Sun 19 Mar 2023 19:04:46 GMT" www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> If-Modified-Since: Sun, 19 Mar 2023 19:04:46 GMT
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 304 Not Modified
< Date: Sun, 30 Aug 2026 19:31:24 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
<
* Connection #0 to host www.redes.unlp.edu.ar left intact
redes@debian:~$
```

Utilizo el parámetro `-z` seguido de una fecha **posterior** a la fecha original. Lo que hace este parámetro es inyectar automáticamente la cabecera HTTP `If-Modified-Since` en la petición GET. Es fundamental agregar el parámetro `-v` (verbose) porque es la única forma de visualizar el intercambio de cabeceras y el código de estado en la terminal. El servidor evalúa la petición, compara las fechas y verifica que el archivo no sufrió modificaciones posteriores a la fecha enviada. En consecuencia, responde con un código de estado `HTTP/1.1 304 Not Modified`. En esta respuesta, el servidor devuelve las cabeceras pero omite enviar el cuerpo del mensaje, razón por la cual la terminal no imprime la estructura de la página. Permite que un cliente (como el navegador web) le pregunte al servidor: "Ya tengo una copia de este recurso desde esta fecha, ¿hubo algún cambio nuevo?". Si el servidor responde con un `304 Not Modified`, el navegador sabe que puede renderizar su copia local de forma segura. Reduciendo la carga de procesamiento del lado del servidor, acelerando los tiempos de respuesta.

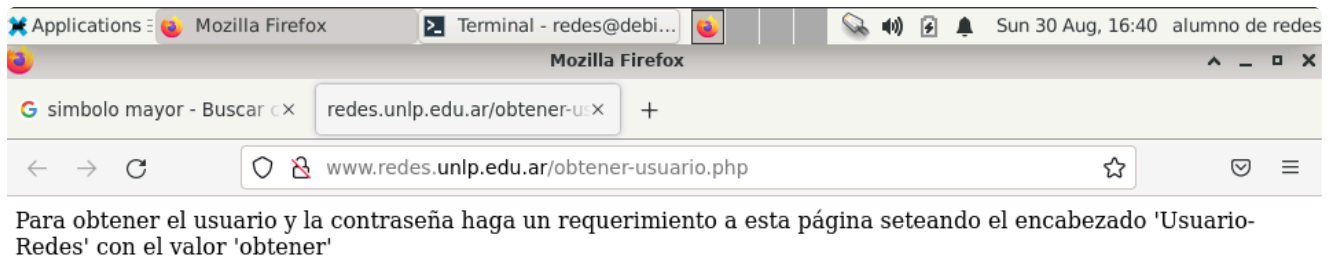
Inciso 14

Utilizando curl, acceda al sitio www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados

Primer paso:

```
redes@debian:~$ curl www.redes.unlp.edu.ar/restringido/index.php
<h1>Acceso restringido</h1>

<p>Para acceder al contenido es necesario autenticarse. Para obtener los datos de acceso seguir las instrucciones detalladas en www.redes.unlp.edu.ar/obtener-usuario.php</p>
redes@debian:~$
```



Segundo paso:

```
redes@debian:~$ curl -H "Usuario-Redes:obtener" www.redes.unlp.edu.ar/obtener-usuario.php
<p>Bien hecho! Los datos para ingresar son:

    Usuario: redes

    Contraseña: RYC

    Ahora vuelva a acceder a la página inicial con los datos anteriores.

    PISTA: Investigue el uso del encabezado Authorization para el método Basic. El comando base64 puede ser de ayuda!</p>
redes@debian:~$
```

Utilizamos el parámetro -H para poder agregar el encabezado que nos pide la página.

Tercer paso

Para poder autenticarnos el protocolo HTTP Basic exige que unamos el nombre de usuario y la contraseña con dos puntos (:), sin espacios. Utilizaremos base 64 para poder codificar lo que queremos enviar

```
redes@debian:~$ echo -n "redes:RYC" | base64
cmVkZXZM6U11D
```

Cuarto paso

Utilizar -H para agregar la encabezado authorization

```
redes@debian:~$ curl -H "Authorization: Basic cmVkZXM6UllD" www.redes.unlp.edu.ar/restringido/index.php
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>

redes@debian:~$
```

Quinto paso

```
redes@debian:~$ curl -v -H "Authorization: Basic cmVkZXM6UllD" www.redes.unlp.edu.ar/restringido/index.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/index.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXM6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 302 Found
< Date: Sun, 30 Aug 2026 19:53:58 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
< Content-Length: 230
< Content-Type: text/html; charset=UTF-8
<
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>

* Connection #0 to host www.redes.unlp.edu.ar left intact
redes@debian:~$
```

Utilizo el parámetro -v para ver qué me devuelve la respuesta, además del contenido, para saber donde debo entregar los datos. También se podría usar -i o -l luego de ya saber que nos devuelve.

Sexto paso

```
redes@debian:~$ curl -v -H "Authorization: Basic cmVkZXM6UllD" http://www.redes.unlp.edu.ar/restringido/the-end.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/the-end.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXM6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sun, 30 Aug 2026 19:56:58 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Content-Length: 159
< Content-Type: text/html; charset=UTF-8
<
¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2026-08-30 19:56:58
* Connection #0 to host www.redes.unlp.edu.ar left intact
Verificación: cb749c4c4dbe4f6a4ecaac3e83e1ce74fc25fdc2b56f2bc493c4c4b60cc231f6redes@debian:~$
```

Inciso 15

Utilizando la VM, realice las siguientes pruebas:

1. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).
2. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?
3. Repita el proceso anterior, pero copiando la salida del recurso /extras/prueba-http-1-1.txt. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.

Inciso 15.1

```
redes@debian:~$ curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

redes@debian:~$
```

Inciso 15.2

```
redes@debian:~$ telnet www.redes.unlp.edu.ar 80
Trying 172.28.0.50...
Connected to www.redes.unlp.edu.ar.
Escape character is '^]'.
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 20:58:45 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "760-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Connection: close
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Protocolo HTTP: versiones</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">
    <!-- Le styles -->
    <link href="../../bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="../../css/style.css" rel="stylesheet">
    <link href="../../bootstrap/css/bootstrap-responsive.css" rel="stylesheet">
```

Luego de hacerlo devuelve un código html de una página.

Inciso 15.3

```
redes@debian:~$ curl www.redes.unlp.edu.ar/extras/prueba-http-1-1.txt
GET /http/HTTP-1.1/ HTTP/1.1
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

```
redes@debian:~$
```

```
redes@debian:~$ telnet www.redes.unlp.edu.ar 80
Trying 172.28.0.50...
Connected to www.redes.unlp.edu.ar.
Escape character is '^J'.
GET /http/HTTP-1.1/ HTTP/1.1
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

```
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 21:03:02 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "760-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Content-Type: text/html
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Protocolo HTTP: versiones</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">
```

```
  <!-- Le styles -->
```

```
    <p class="muted credit">Redes y Comunicaciones</p>
```

```
  </div>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
GET /http/HTTP-1.1/ HTTP/1.1
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

```
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 21:03:49 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "760-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Content-Type: text/html
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Protocolo HTTP: versiones</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="../../bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="../../css/style.css" rel="stylesheet">
    <link href="../../bootstrap/css/bootstrap-responsive.css" rel="stylesheet">
```

Fue posible volver a pegar el mensaje porque al ser HTTP 1.1, la conexión es persistente por defecto.

Inciso 16

En base a lo obtenido en el ejercicio anterior, responda:

1. ¿Qué está haciendo al ejecutar el comando telnet?
2. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?
3. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?
4. ¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?

Inciso 16.1

El comando telnet se utiliza para comunicarse e forma interactiva con otro host, con el protocolo telnet.

Inciso 16.2

En ambos utilizó el método **GET**. Solicitó el recurso `/http/HTTP-1.1/` (especificando la versión del protocolo HTTP correspondiente en cada caso, `HTTP/1.0` en el inciso 15.1 y `HTTP/1.1` en el 15.3)

Inciso 16.3

- **Caso 1 (primer petición):** Tras enviar la petición y recibir la respuesta del servidor, **la conexión TCP se cierra automáticamente** (lo podemos notar en la respuesta del servidor con la cabecera `Connection: close`).
- **Caso 2(segunda petición):** Utiliza **conexiones persistentes** (por defecto mediante Keep-Alive). Esto significa que una vez establecida la conexión TCP y recibida la primera respuesta, el canal queda abierto, permitiéndote pegar y enviar múltiples solicitudes adicionales sin necesidad de volver a ejecutar el comando `telnet` para reconectarte.

Inciso 16.4

El segundo caso sería mas eficiente en caso de requerir más archivos en el momento, como en el caso del ejercicio mencionado. En el caso del primero,

tendríamos que reabrir varias veces la conexión, en caso de requerir más recursos.

Aunque HTTP/1.1 con conexiones persistentes soluciona el problema de reconexión constante, sufre de **bloqueo de cabeza de línea (Head-of-Line Blocking)** a nivel de aplicación. En HTTP/1.1, aunque la conexión TCP se mantiene abierta (persistente), las peticiones se procesan de forma estrictamente **secuencial**, si una solicitud se demora en responder, las solicitudes que están en cola esperando por la misma conexión se quedan trabadas hasta que se libere el canal

Inciso 17

En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark. Antes de iniciar considere:

- Capture los paquetes utilizando la interfaz con IP 172.28.0.1. (Menú “Capture ->Options”. Luego seleccione la interfaz correspondiente y presione Start).
- Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena ‘http’ (sin las comillas) en la ventana de especificación de filtros de visualización (display-filter). Si no hiciéramos esto veríamos todo el tráfico que es capaz de capturar nuestra placa de red. De los paquetes que son capturados, aquel que esté seleccionado será mostrado en forma detallada en la sección que está justo debajo. Como sólo estamos interesados en http ocultaremos toda la información que no es relevante para esta práctica (Información de trama, Ethernet, IP y TCP). Desplegar la información correspondiente al protocolo HTTP bajo la leyenda “Hypertext Transfer Protocol”.
- Para borrar la cache del navegador, deberá ir al menú “Herramientas->Borrar historial reciente”. Alternativamente puede utilizar Ctrl+F5 en el navegador para forzar la petición HTTP evitando el uso de caché del navegador.
- En caso de querer ver de forma simplificada el contenido de una comunicación http, utilice el botón derecho sobre un paquete HTTP perteneciente al flujo capturado y seleccione la opción Follow TCP Stream.

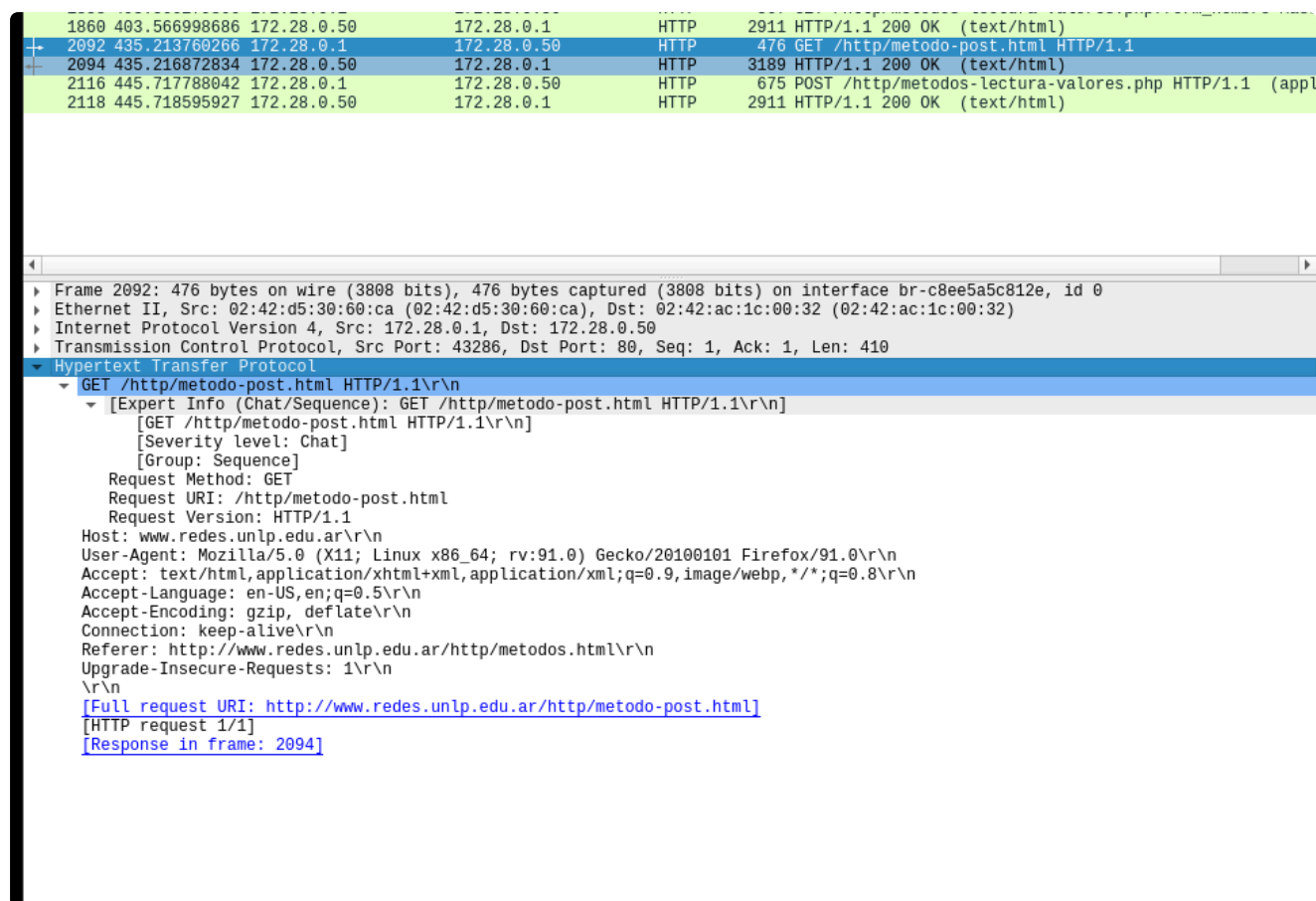
1. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección “Capa de Aplicación” llamado “Métodos HTTP”. En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario como el siguiente:



2. Analice el código HTML
3. Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.
4. ¿Qué diferencias detectó en los mensajes enviados por el cliente?
5. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?

Respuesta

Petición GET:



No.	Time	Source	Destination	Protocol	Length	Info
1860	403.566998686	172.28.0.50	172.28.0.1	HTTP	2911	HTTP/1.1 200 OK (text/html)
2092	435.213760266	172.28.0.1	172.28.0.50	HTTP	476	GET /http/metodo-post.html HTTP/1.1
2094	435.216872834	172.28.0.50	172.28.0.1	HTTP	3189	HTTP/1.1 200 OK (text/html)
2116	445.717788042	172.28.0.1	172.28.0.50	HTTP	675	POST /http/metodos-lectura-valores.php HTTP/1.1 (appli
2118	445.718595927	172.28.0.50	172.28.0.1	HTTP	2911	HTTP/1.1 200 OK (text/html)

Frame 2092: 476 bytes on wire (3808 bits), 476 bytes captured (3808 bits) on interface br-c8ee5a5c812e, id 0
Ethernet II, Src: 02:42:d5:30:60:ca (02:42:d5:30:60:ca), Dst: 02:42:ac:1c:00:32 (02:42:ac:1c:00:32)
Internet Protocol Version 4, Src: 172.28.0.1, Dst: 172.28.0.50
Transmission Control Protocol, Src Port: 43286, Dst Port: 80, Seq: 1, Ack: 1, Len: 410
Hypertext Transfer Protocol
GET /http/metodo-post.html HTTP/1.1\r\n
[Expert Info (Chat/Sequence): GET /http/metodo-post.html HTTP/1.1\r\n]
[GET /http/metodo-post.html HTTP/1.1\r\n]
[Severity level: Chat]
[Group: Sequence]
Request Method: GET
Request URI: /http/metodo-post.html
Request Version: HTTP/1.1
Host: www.redes.unlp.edu.ar\r\nUser-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\nAccept-Language: en-US,en;q=0.5\r\nAccept-Encoding: gzip, deflate\r\nConnection: keep-alive\r\nReferer: http://www.redes.unlp.edu.ar/http/metodos.html\r\nUpgrade-Insecure-Requests: 1\r\n\r\n[Full request URI: http://www.redes.unlp.edu.ar/http/metodo-post.html]
[HTTP request 1/1]
[Response in frame: 2094]

Petición Post:

```
Transmission Control Protocol, Src Port: 56624, Dst Port: 80, Seq: 1, Ack: 1, Len: 609
Hypertext Transfer Protocol
  POST /http/metodos-lectura-valores.php HTTP/1.1\r\n
    [Expert Info (Chat/Sequence): POST /http/metodos-lectura-valores.php HTTP/1.1\r\n]
    [POST /http/metodos-lectura-valores.php HTTP/1.1\r\n]
    [Severity level: Chat]
    [Group: Sequence]
    Request Method: POST
    Request URI: /http/metodos-lectura-valores.php
    Request Version: HTTP/1.1
    Host: www.redes.unlp.edu.ar\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Content-Type: application/x-www-form-urlencoded\r\n
    Content-Length: 76\r\n
    [Content length: 76]
    Origin: http://www.redes.unlp.edu.ar\r\n
    Connection: keep-alive\r\n
    Referer: http://www.redes.unlp.edu.ar/http/metodo-post.html\r\n
    Upgrade-Insecure-Requests: 1\r\n
    \r\n
    [Full request URI: http://www.redes.unlp.edu.ar/http/metodos-lectura-valores.php]
    [HTTP request 1/1]
    [Response in frame: 2118]
    File Data: 76 bytes
HTML Form URL Encoded: application/x-www-form-urlencoded
```

Inciso 17.4

- **Método POST:** Los datos del formulario se envían encapsulados dentro del **cuerpo (body)** del mensaje HTTP. Por este motivo, la cabecera de la petición incluye obligatoriamente los campos `Content-Length` y `Content-Type` (que indica el formato, ej: `application/x-www-form-urlencoded`).

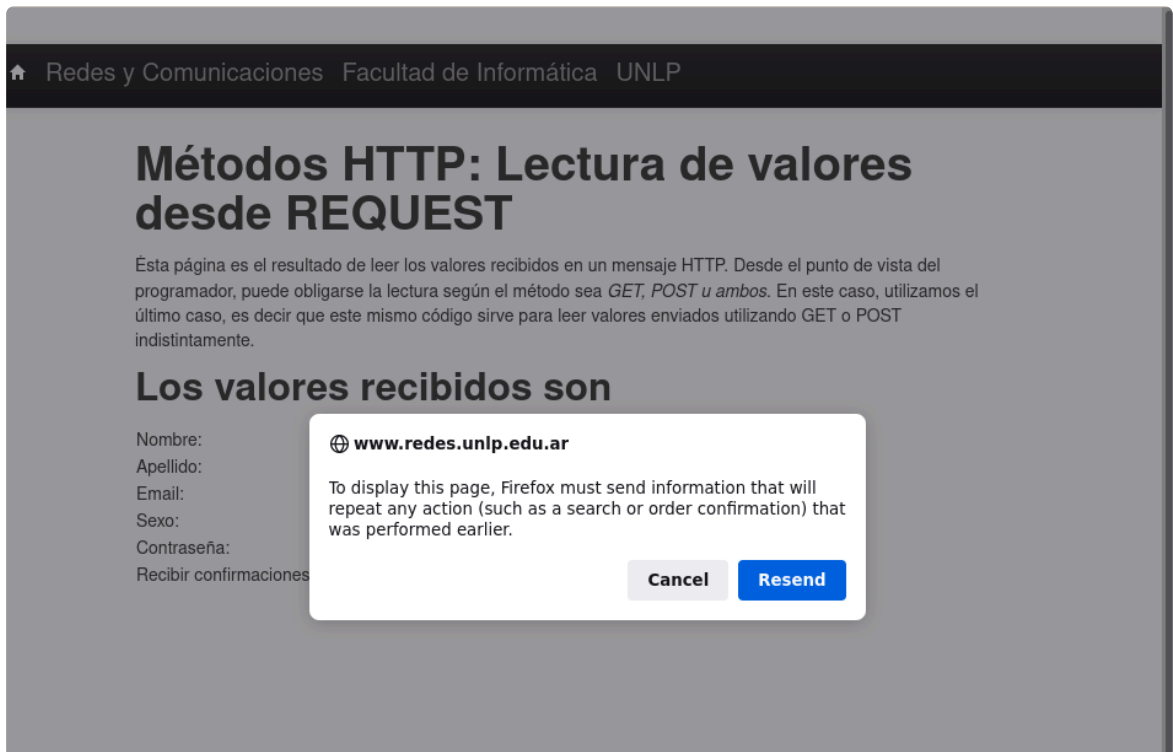
```
Internet Protocol Version 4, Src: 1/2.28.0.1, Dst: 1/2.28.0.50
Transmission Control Protocol, Src Port: 57678, Dst Port: 80, Seq: 1, Ack: 1, Len: 609
Hypertext Transfer Protocol
  HTML Form URL Encoded: application/x-www-form-urlencoded
    Form item: "form_nombre" = "Massimo"
      Key: form_nombre
      Value: Massimo
    Form item: "form_apellido" = ""
      Key: form_apellido
      Value:
    Form item: "form_mail" = ""
      Key: form_mail
      Value:
    Form item: "form_sexo" = "sexo_masc"
    Form item: "form_pass" = ""
```

- **Método GET:** Los datos del formulario se codifican y se adjuntan directamente en la **URI** de la petición, en la primera línea del mensaje. Como el mensaje no contiene cuerpo, no existen los encabezados `Content-Length` ni `Content-Type`.

Inciso 17.5

- **Visibilidad en la barra de direcciones:** Al utilizar el método GET, todos los parámetros y valores ingresados en el formulario quedan explícitamente visibles en la URL del navegador. Con el método POST, los parámetros viajan de forma oculta y la URL permanece limpia.

- **Comportamiento al recargar (F5):** Si se intenta actualizar la página después de haber enviado el formulario mediante POST, el navegador interrumpe la acción con una alerta de seguridad solicitando confirmación para "reenviar los datos del formulario". Si el envío se realizó por GET, la recarga se ejecuta inmediatamente sin mostrar ninguna advertencia.



Inciso 18

Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.

Respuesta:

Por diseño, un servidor HTTP no mantiene ninguna información sobre las solicitudes pasadas de un cliente; es decir, **HTTP es un protocolo sin memoria de estado**.

Para simplificar el diseño de los servidores pero al mismo tiempo permitir funciones complejas, las cookies introducen una capa de sesión sobre el protocolo HTTP, permitiendo identificar y seguir la pista de los usuarios.

El sistema de cookies funciona mediante la interacción de cuatro componentes:

1. **La cabecera Set-Cookie:** incluida en el mensaje de respuesta HTTP (enviada por el servidor).
2. **La cabecera Cookie:** incluida en el mensaje de solicitud HTTP (enviada por el cliente de forma automática).
3. **Un archivo de cookies** almacenado en la máquina del usuario y gestionado por su propio navegador.
4. **Una base de datos back-end** en el sitio web del servidor que asocia el identificador único del usuario con su actividad en el sitio.

Inciso 19

¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?

Respuesta

Basados en texto	Binarios
Los mensajes se transmiten en un formato de caracteres que las personas pueden leer directamente (texto ASCII plano). Cada campo y línea de cabecera se separa mediante delimitadores específicos, como espacios en blanco o secuencias de retorno de carro y salto de línea (CRLF). Su legibilidad facilita la depuración y las pruebas manuales	Los datos se transmiten estructurados en campos estrictos de bits y bytes diseñados específicamente para que las computadoras los interpreten sin necesidad de traducir texto. No son legibles por seres humanos de forma directa

HTTP/1.0 y HTTP/1.1, son basados en texto. Sus mensajes de solicitud y respuesta consisten enteramente en líneas de texto ASCII legible.

HTTP/2 es un protocolo binario. Mantiene la misma semántica que HTTP/1.1, pero introduce la capa de capa de encuadre binario (binary framing layer). Esta capa rompe los mensajes tradicionales de texto plano y los convierte en "tramas" (frames) binarias discretas e independientes.

Inciso 20

Responder las siguientes preguntas:

1. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2? (Ayuda: <https://undertow.io/blog/2015/04/27/An-in-deph-overview-of-HTTP2.html> para HTTP/2)
2. En HTTP/1.1, ¿es correcto el siguiente requerimiento?
GET /index.php HTTP/1.1
User-Agent: curl/7.54.0
3. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?
GET /index.php HTTP/1.1
Host: www.info.unlp.edu.ar

Inciso 20.1

En HTTP/1.1 especifica el nombre de dominio del servidor al que se destina la petición. Esto es fundamental para el **Alojamiento Virtual (Virtual Hosting)**, que permite a un único servidor físico (con una sola dirección IP) alojar múltiples sitios web distintos.

En HTTP/1.0 no existía de forma oficial en la especificación original. En HTTP/1.0 no era obligatoria. Sin embargo, debido a que el "Virtual Hosting" se volvió indispensable para el crecimiento de la Web, se terminó adoptando informalmente como un agregado compatible en HTTP/1.0 antes de convertirse en un estándar estrictamente obligatorio en HTTP/1.1.

En HTTP/2, las cabeceras tradicionales basadas en texto se transforman. La cabecera Host es reemplazada oficialmente por una pseudo-cabecera (pseudo-header) que comienza con dos puntos: :authority.

Inciso 20.2

En HTTP/1.1, la cabecera Host es obligatoria. Si un cliente envía un requerimiento HTTP/1.1 sin ella, el servidor tiene la obligación de rechazar la solicitud

Inciso 20.3

En HTTP/2 no existe la línea de solicitud tradicional (como GET /index.php HTTP/1.1). En su lugar, el mensaje es troceado en tramas binarias y los

campos de la línea de solicitud y la cabecera `Host` se mapean como **pseudo-cabeceras**

1	
2	
3	
4	